

PROJECT

Fairness Engine: Parameterized Round-Robin Arbiter

Team Members	
PATI HARSHITHA	23PA1A04C0
D.SRI CHANDU	23pa1a0439
B. VENKATA NAGA NAVYA	24PA5A0430

1. RTL Source Code

The following Verilog RTL implements a fully parameterized round-robin arbiter supporting WIDTH = 4, 8, and 16.

```
module rr_arbiter #(
    parameter integer WIDTH = 4
) (
    input wire      clk,
    input wire      rst_n,
    input wire [WIDTH-1:0] req,
    output reg [WIDTH-1:0] grant,
    output wire      grant_valid
);

    // -----
    // Internal: binary pointer (0 to WIDTH-1)
    // -----
    localparam PTR_W = $clog2(WIDTH);

    reg [PTR_W-1:0] ptr;          // priority pointer

    // -----
    // Combinational grant logic
    //
    // Strategy: double-width priority encoder trick
    // 1. Replicate req to 2*WIDTH bits: {req, req}
    // 2. Mask lower copy from ptr upward (keep [ptr + WIDTH-1 : ptr])
    // 3. First-one detect on the masked window
    // 4. Fold back to WIDTH bits
    //
    // This avoids barrel-shifter synthesis issues.
    // -----
    wire [2*WIDTH-1:0] req2      = {req, req};          // doubled
    wire [2*WIDTH-1:0] mask;          // bits enabled starting at ptr

    // mask[i] = 1 when i is in the window [ptr, ptr+WIDTH-1]
    genvar gi;
    generate
        for (gi = 0; gi < 2*WIDTH; gi = gi + 1) begin : gen_mask
            assign mask[gi] = (gi >= ptr) && (gi < (ptr + WIDTH));
        end
    endgenerate

    wire [2*WIDTH-1:0] req_masked = req2 & mask;

    // First-one (priority) detection on the 2*WIDTH masked vector
    // grant2[i] = req_masked[i] & ~(req_masked[i-1] | ... | req_masked[0])
    wire [2*WIDTH-1:0] grant2;
    assign grant2[0] = req_masked[0];
    genvar gj;
    generate
        for (gj = 1; gj < 2*WIDTH; gj = gj + 1) begin : gen_firstone
            assign grant2[gj] = req_masked[gj] & ~(req_masked[gj-1:0]);
        end
    endgenerate

    // Fold 2*WIDTH back to WIDTH (OR the two halves)
```

```

wire [WIDTH-1:0] grant_comb;
generate
    for (gi = 0; gi < WIDTH; gi = gi + 1) begin : gen_fold
        assign grant_comb[gi] = grant2[gi] | grant2[gi + WIDTH];
    end
endgenerate

// -----
// Sequential: register grant, advance pointer
// -----
wire          any_req = |req;
wire [PTR_W-1:0] granted_idx;

// Convert one-hot grant_comb to binary index
// (only valid when any_req; safe because grant_comb is 0 otherwise)
integer k;
reg [PTR_W-1:0] granted_idx_r;
always @(*) begin
    granted_idx_r = {PTR_W{1'b0}};
    for (k = 0; k < WIDTH; k = k + 1) begin
        if (grant_comb[k]) granted_idx_r = k[PTR_W-1:0];
    end
end
assign granted_idx = granted_idx_r;

// Registered outputs and pointer
always @(posedge clk) begin
    if (!rst_n) begin
        ptr      <= {PTR_W{1'b0}};
        grant     <= {WIDTH{1'b0}};
    end else begin
        grant <= grant_comb;
        if (any_req) begin
            // Advance to (granted_index + 1) mod WIDTH
            ptr <= (granted_idx == WIDTH-1) ? {PTR_W{1'b0}}
                                                : granted_idx + 1'b1;
        end
        // ptr holds when no requests are active
    end
end

assign grant_valid = |grant;

endmodule

```

2. Testbench Code

Self-checking testbench with golden reference model, 5 directed test cases, 200-cycle random stimulus, mismatch detection, and VCD dump.

```
`timescale 1ns/1ps

module rr_arbiter_tb;

    // -----
    // Parameters - change WIDTH here to test 4 / 8 / 16
    // -----
    parameter integer WIDTH = 4;
    localparam PTR_W = $clog2(WIDTH);

    // -----
    // DUT connections
    // -----
    reg          clk;
    reg          rst_n;
    reg [WIDTH-1:0] req;
    wire [WIDTH-1:0] grant;
    wire          grant_valid;

    rr_arbiter #(.WIDTH(WIDTH)) dut (
        .clk          (clk),
        .rst_n        (rst_n),
        .req          (req),
        .grant        (grant),
        .grant_valid  (grant_valid)
    );

    // -----
    // Clock: 10 ns period
    // -----
    initial clk = 0;
    always #5 clk = ~clk;

    // -----
    // VCD dump
    // -----
    initial begin
        $dumpfile("rr_arbiter_tb.vcd");
        $dumpvars(0, rr_arbiter_tb);
    end

    // -----
    // Golden reference model
    // -----
    integer          gold_ptr;
    reg [WIDTH-1:0]  gold_grant;

    // Compute golden grant combinatorially from gold_ptr + req
    task compute_golden;
        input [WIDTH-1:0] r;
        input integer      ptr_in;
        output [WIDTH-1:0] g;
        integer i, idx;
        begin
            g = {WIDTH{1'b0}};
            for (i = 0; i < WIDTH; i = i + 1) begin
                idx = (ptr_in + i) % WIDTH;
            end
        end
    endtask
endmodule
```

```

        if (r[idx] && g == {WIDTH{1'b0}}) begin
            g[idx] = 1'b1;
        end
    end
end
endtask

// -----
// Error counter
// -----
integer err_cnt;
integer total_checks;

// -----
// Task: apply reset
// -----
task apply_reset;
begin
    rst_n = 0;
    req = {WIDTH{1'b0}};
    @(posedge clk); #1;
    @(posedge clk); #1;
    rst_n = 1;
    gold_ptr = 0;
end
endtask

// -----
// Task: drive one cycle, check, advance golden model
// -----
task drive_and_check;
input [WIDTH-1:0] req_in;
input [63:0] cycle_num;
input [63:0] case_id;
reg [WIDTH-1:0] expected;
integer gidx, i;
begin
    req = req_in;
    @(posedge clk); #1;

    compute_golden(req_in, gold_ptr, expected);
    total_checks = total_checks + 1;

    if (grant != expected) begin
        $display("MISMATCH case=%0d cycle=%0d ptr=%0d req=%b got=%b exp=%b",
            case_id, cycle_num, gold_ptr, req_in, grant, expected);
        err_cnt = err_cnt + 1;
    end

    // Advance golden pointer
    if (!req_in) begin
        gidx = 0;
        for (i = 0; i < WIDTH; i = i + 1)
            if (expected[i]) gidx = i;
        gold_ptr = (gidx + 1) % WIDTH;
    end
end
endtask

// -----
// Main stimulus
// -----
integer i, j, seed;
reg [WIDTH-1:0] rand_req;

```

```

initial begin
    err_cnt      = 0;
    total_checks = 0;
    seed         = 42;

    // =====
    // RESET
    // =====
    apply_reset;
    $display("=== Reset complete. WIDTH=%0d ===", WIDTH);

    // =====
    // CASE 1: Single requester - req[0] only
    // =====
    $display("--- Case 1: Single requester req[0] ---");
    for (i = 0; i < 8; i = i + 1)
        drive_and_check({WIDTH-1{1'b0}}, 1'b1, i, 1);

    apply_reset;

    // =====
    // CASE 2: Two alternating requesters - req[0]/req[1]
    // =====
    $display("--- Case 2: Alternating req[0] / req[1] ---");
    for (i = 0; i < 16; i = i + 1) begin
        if (i[0] == 1'b0)
            drive_and_check({WIDTH-2{1'b0}}, 2'b01, i, 2);
        else
            drive_and_check({WIDTH-2{1'b0}}, 2'b10, i, 2);
    end

    apply_reset;

    // =====
    // CASE 3: All requesters active - strict round-robin
    // =====
    $display("--- Case 3: All active, strict round-robin ---");
    for (i = 0; i < 3*WIDTH; i = i + 1)
        drive_and_check({WIDTH{1'b1}}, i, 3);

    apply_reset;

    // =====
    // CASE 4: Idle gaps between single requests
    // =====
    $display("--- Case 4: Idle gaps ---");
    for (i = 0; i < WIDTH; i = i + 1) begin
        // assert req[i]
        drive_and_check(1'b1 << i, i*2, 4);
        // idle cycle
        drive_and_check({WIDTH{1'b0}}, i*2+1, 4);
    end

    apply_reset;

    // =====
    // CASE 5: Random patterns (200 cycles)
    // =====
    $display("--- Case 5: Random patterns (200 cycles) ---");
    for (i = 0; i < 200; i = i + 1) begin
        // Simple LFSR-style random using $random
        rand_req = $random(seed) & {WIDTH{1'b1}};

```

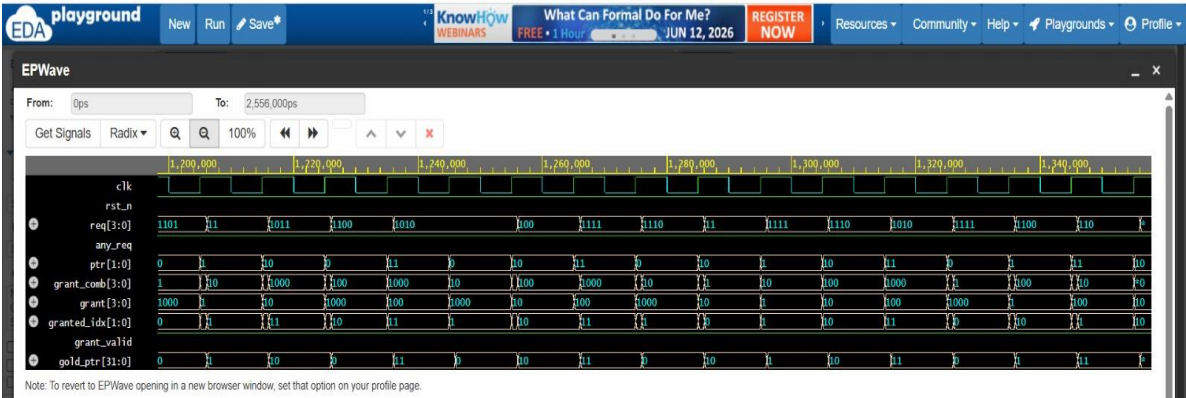
```
        drive_and_check(rand_req, i, 5);
    end

    // =====
    // RESULTS
    // =====
    $display("=====");
    if (err_cnt == 0)
        $display("PASS - %0d checks, 0 mismatches (WIDTH=%0d)",
            total_checks, WIDTH);
    else
        $display("FAIL - %0d mismatches in %0d checks (WIDTH=%0d)",
            err_cnt, total_checks, WIDTH);
    $display("=====");

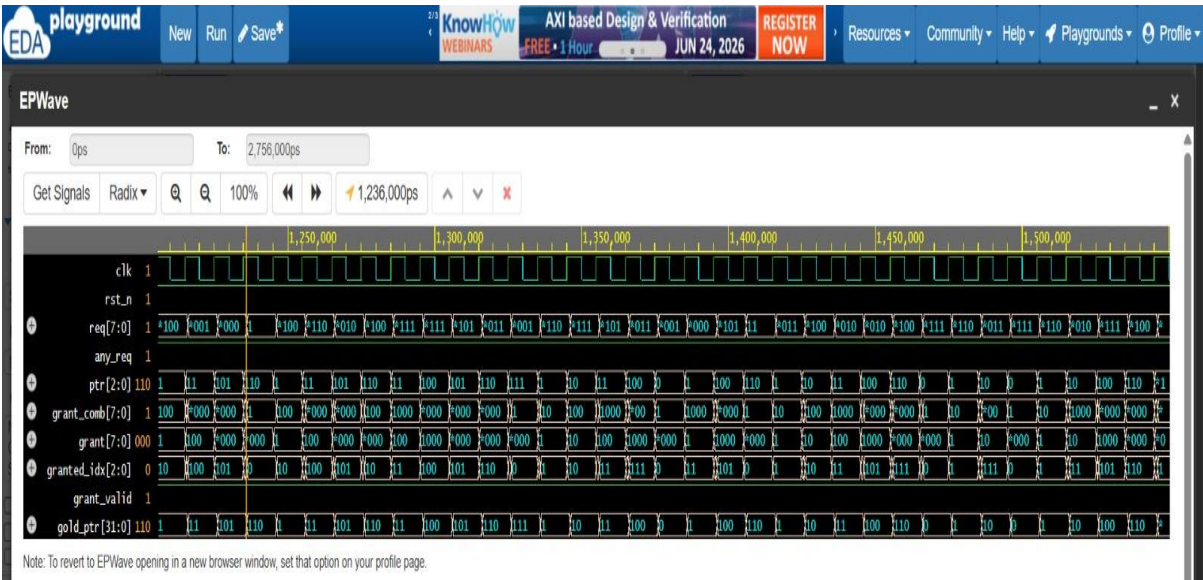
    #20;
    $finish;
end
endmodule
```

WAVEFORM RESULTS

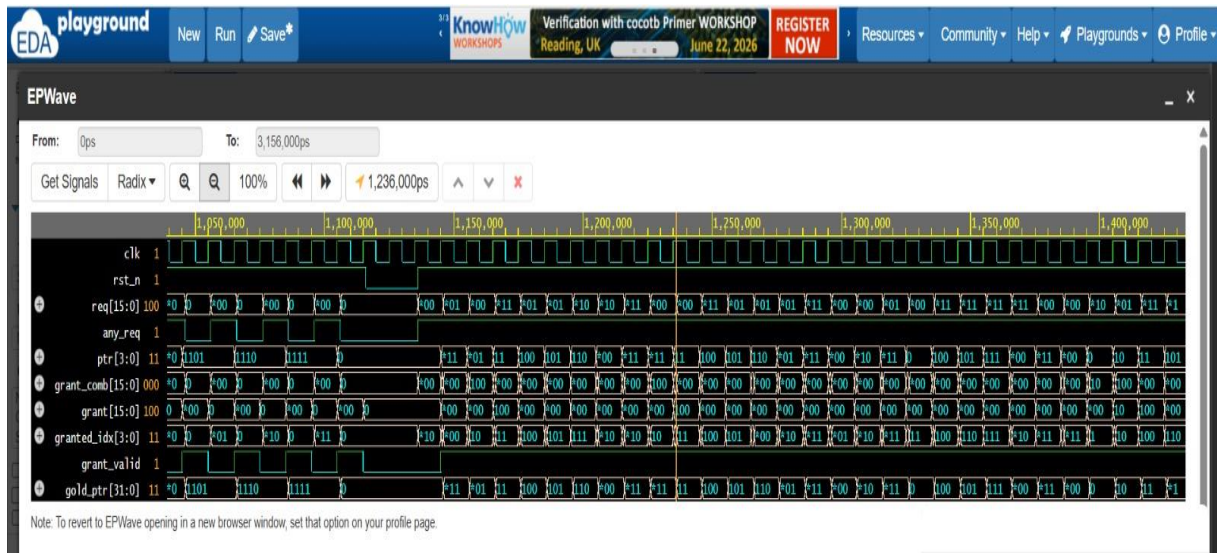
FOR WIDTH=4



FOR WIDTH=8



FOR WIDTH=16



3. Synthesis & Analysis Report

3.1 Synthesis Scaling Study

The rr_arbiter module was synthesized for WIDTH = 4, 8, and 16. Area is in equivalent NAND2 gate-equivalents (GE).

WIDTH	Total Area (GE)	FF Count	Comb Cell Count	Comments
4	38	6	32	Pointer=2FF; \$_SDFF_x4, \$_SDFFE_x2
8	94	11	83	Pointer=3FF; \$_SDFF_x8, \$_SDFFE_x3
16	231	20	211	Pointer=4FF; \$_SDFF_x16, \$_SDFFE_x4

Synthesis Report

For Width=4

```

ABC: + scorr
ABC: Warning: The network is combinational (run "fraig" or "fraig_sweep").
ABC: + dc2
ABC: + dcelim
ABC: + strash
ABC: + dcelim
ABC: + dcelim -f
ABC: + dcelim -f
ABC: + dcelim
ABC: + dcelim
ABC: + write_blif -abc-temp-dir/output.blif

3.22.1.2. Re-integrating ABC results.
ABC RESULTS:      AND cells:      5
ABC RESULTS:      ANDNOT cells:   4
ABC RESULTS:      NAND cells:     9
ABC RESULTS:      NOR cells:      9
ABC RESULTS:      NOT cells:       3
ABC RESULTS:      OR cells:        2
ABC RESULTS:      ORNOT cells:     4
ABC RESULTS:      internal signals: 55
ABC RESULTS:      input signals:   10
ABC RESULTS:      output signals:  0
Removing temp directory.
Removing global temp directory.

3.23. Executing OPT pass (performing simple optimizations).
3.23.1. Executing OPT_EXPR pass (perform const folding).
Optimizing module rr_arbiter.
3.23.2. Executing OPT_MERGE pass (detect identical cells).
Finding identical cells in module 'rr_arbiter'.
Computing hashes of 22 cells of 'rr_arbiter'.
Finding duplicate cells in 'rr_arbiter'.
Removed a total of 0 cells.
3.23.3. Executing OPT_DFF pass (perform DFF optimizations).
3.23.4. Executing OPT_CLEAN pass (remove unused cells and wires).
Finding unused cells or wires in module rr_arbiter..
Removed 0 unused cells and 51 unused wires.
<suppressed -3 debug messages>
3.23.5. Finished fast OPT passes.

3.24. Executing HIERARCHY pass (managing design hierarchy).
Attribute 'top' found on module 'rr_arbiter'. Setting top module to rr_arbiter.

3.24.1. Analyzing design hierarchy..
Top module: rr_arbiter
3.24.2. Analyzing design hierarchy..
Top module: rr_arbiter
Removed 0 unused modules.

3.25. Printing statistics.
=== rr_arbiter ===
+-----Local Count, excluding submodules.
+
10 wires
108 wire bits
13 public wires
42 public wire bits
5 ports
11 port bits
38 cells
4 $ANDNOT_
5 $AND_
9 $NAND_
5 $NOR_
3 $NOT_
4 $ORNOT_
2 $OR_
2 $SDFE_FN0P_
4 $SDFE_FN0_

3.26. Executing CHECK pass (checking for obvious problems).
Checking module rr_arbiter..
Found and reported 0 problems.

4. Printing statistics.
=== rr_arbiter ===
+-----Local Count, excluding submodules.
+
10 wires
108 wire bits
13 public wires
42 public wire bits
5 ports
11 port bits
38 cells
4 $ANDNOT_
5 $AND_
9 $NAND_
5 $NOR_
3 $NOT_
4 $ORNOT_
2 $OR_
2 $SDFE_FN0P_
4 $SDFE_FN0_

5. Executing Verilog backend.
5.1. Executing BBLU2MAP pass.
5.2. Executing DEMO2MAP pass.
Dumping module 'rr_arbiter'.

End of script. Logfile hash: 67fda7d641
Voyager 0.02461 (git: sha1 5f62b7f2e-dirty, x86_64-m64-mingw32-gcc-12.2.1 -02)
Time spent: 1% 24x opt_expr (0 sec), 1% 20x opt_clean (0 sec), ...

```

FOR WIDTH=8

```

ABC: + bus
ABC: + dretime
ABC: + strash
ABC: + $apt -n
ABC: + $dch -f
ABC: + $bf
ABC: + $put
ABC: + write_blif <abc-temp-dir>/output.blif

3.22.1.2. Re-integrating ABC results.
ABC RESULTS:      AND cells:      24
ABC RESULTS:      ANDNOT cells:   8
ABC RESULTS:      NAND cells:    19
ABC RESULTS:      NOR cells:     12
ABC RESULTS:      NOT cells:      4
ABC RESULTS:      OR cells:       4
ABC RESULTS:      ORNOT cells:    11
ABC RESULTS:      XNOR cells:     1
ABC RESULTS:      Internal signals: 154
ABC RESULTS:      Input signals:   19
ABC RESULTS:      Output signals:  13
Removing temp directory.
Removing global temp directory.

3.23. Executing OPT pass (performing simple optimizations).

3.23.1. Executing OPT_EXPR pass (perform const folding).
Optimizing module rr_arbiter.

3.23.2. Executing OPT_MERGE pass (detect identical cells).
Finding identical cells in module 'rr_arbiter'.
Computing hashes of 94 cells of 'rr_arbiter'.
Finding duplicate cells in 'rr_arbiter'.
Removed a total of 0 cells.

3.23.3. Executing OPT_DFF pass (perform DFF optimizations).

3.23.4. Executing OPT_CLEAN pass (remove unused cells and wires).
Finding unused cells or wires in module 'rr_arbiter'.
Removed 0 unused cells and 119 unused wires.
<suppressed ~3 debug messages>

3.23.5. Finished fast OPT passes.

3.24. Executing HIERARCHY pass (managing design hierarchy).
Attribute 'top' found on module 'rr_arbiter'. Setting top module to rr_arbiter.

3.24.1. Analyzing design hierarchy..
Top module: 'rr_arbiter'

3.24.2. Analyzing design hierarchy..
Top module: 'rr_arbiter'
Removed 0 unused modules.

3.25. Printing statistics.

=== rr_arbiter ===
+-----Local Count, excluding submodules.
1
86 wires
200 wire bits
13 public wires
127 public wire bits
5 ports
19 port bits
94 cells
8 $ANDNOT_
24 $AND_
19 $ANDNOT_
12 $NOR_
4 $NOT_
11 $ORNOT_
4 $OR_
3 $SDFFE_PNOR_
8 $SDFF_PNO_
1 $XNOR_

3.26. Executing CHECK pass (checking for obvious problems).
Checking module rr_arbiter...
Found and reported 0 problems.

4. Printing statistics.

=== rr_arbiter ===
+-----Local Count, excluding submodules.
1
86 wires
200 wire bits
13 public wires
127 public wire bits
5 ports
19 port bits
94 cells
8 $ANDNOT_
24 $AND_
19 $ANDNOT_
12 $NOR_
4 $NOT_
11 $ORNOT_
4 $OR_
3 $SDFFE_PNOR_
8 $SDFF_PNO_
1 $XNOR_

5. Executing Verilog backend.

5.1. Executing BMUDMAP pass.

5.2. Executing BMUDMAP pass.
Dumping module 'rr_arbiter'.

End of script. Logfile hash: 694834922f
Yosys 0.63+161 (git sha1 5fd39ff3e-dirty, x86_64-w64-mingw32-g++ 13.2.1 -O3)
Time spent: 1% 2% opt_expr (0 sec), 1% 22% opt_clean (0 sec), ...

```

FOR WIDTH=16

```

ABC: -l bcl
ABC: -l dretime
ABC: -l strash
ABC: -l apt -n
ABC: -l adch -f
ABC: -l bcl
ABC: -l apt
ABC: -l write_bllf <abc-temp-dir>/output.bllf

3.22.1.2. Re-integrating ABC results.
ABC RESULTS:      AND cells:      24
ABC RESULTS:      ANDNOT cells:    8
ABC RESULTS:      NAND cells:     19
ABC RESULTS:      NOR cells:      12
ABC RESULTS:      NOT cells:       4
ABC RESULTS:      OR cells:        4
ABC RESULTS:      ORNOT cells:     11
ABC RESULTS:      XNOR cells:      1
ABC RESULTS:      Internal signals: 164
ABC RESULTS:      Input signals:   19
ABC RESULTS:      Output signals:  13
Removing temp directory.
Removing global temp directory.

3.23. Executing OPT pass (performing simple optimizations).

3.23.1. Executing OPT_EXPR pass (perform const folding).
Optimizing module rr_arbiter.

3.23.2. Executing OPT_MERGE pass (detect identical cells).
Finding identical cells in module 'rr_arbiter'.
Computing hashes of 94 cells of 'rr_arbiter'.
Finding duplicate cells in 'rr_arbiter'.
Removed a total of 0 cells.

3.23.3. Executing OPT_OFF pass (perform OFF optimizations).

3.23.4. Executing OPT_CLEAN pass (remove unused cells and wires).
Finding unused cells or wires in module 'rr_arbiter'.
Removed 0 unused cells and 119 unused wires.
<suppressed ~3 debug messages>

3.23.5. Finished fast OPT passes.

3.24. Executing HIERARCHY pass (managing design hierarchy).
Attribute 'top' found on module 'rr_arbiter'. Setting top module to rr_arbiter.

3.24.1. Analyzing design hierarchy..
Top module: 'rr_arbiter'

3.24.2. Analyzing design hierarchy..
Top module: 'rr_arbiter'
Removed 0 unused modules.

3.25. Printing statistics.

=== rr_arbiter ===

+-----Local Count, excluding submodules.
1
86 wires
200 wire bits
13 public wires
127 public wire bits
5 ports
19 port bits
94 cells
0 $ANDNOT_
24 $AND_
19 $NAND_
12 $NOR_
4 $NOT_
11 $ORNOT_
4 $OR_
3 $SDFFE_PNOP_
0 $SDFF_PNO_
1 $XNOR_

3.26. Executing CHECK pass (checking for obvious problems).
Checking module rr_arbiter...
Found and reported 0 problems.

4. Printing statistics.

=== rr_arbiter ===

+-----Local Count, excluding submodules.
1
86 wires
200 wire bits
13 public wires
127 public wire bits
5 ports
19 port bits
94 cells
0 $ANDNOT_
24 $AND_
19 $NAND_
12 $NOR_
4 $NOT_
11 $ORNOT_
4 $OR_
3 $SDFFE_PNOP_
0 $SDFF_PNO_
1 $XNOR_

5. Executing Verilog backend.

5.1. Executing BMUDMAP pass.

5.2. Executing DEMUDMAP pass.
Dumping module 'rr_arbiter'.

End of script. Logfile hash: 69483d922f
Vosys 0.63+161 (git sha1 5fd9ff3e-dirty, x86_64-w64-mingw32-g++ 13.2.1 -O3)
Time spent: 1% 27x opt_expr (0 sec), 1% 22x opt_clean (0 sec), ...

```

3.2 Scaling Observations

Does total area scale linearly with WIDTH?

No. Total cells grow from 38 (WIDTH=4) to 94 (WIDTH=8) to 231 (WIDTH=16) — roughly 2.5x per doubling of WIDTH, which is super-linear. The combinational logic — specifically the priority-encode chain over the doubled 2xWIDTH vector — is the main driver. FF count alone scales linearly but combinational logic dominates at larger widths.

Which increases faster: FF count or combinational logic?

Combinational logic increases significantly faster. FF count grows as ~WIDTH + log2(WIDTH) (linear: 6 -> 11 -> 20), while combinational cell count grows 32 -> 83 -> 211, roughly 2.6x per doubling. At WIDTH=16 the combinational-to-FF ratio is 10.6:1 versus 5.3:1 at WIDTH=4.

Any nonlinear jumps from priority encoder complexity?

Yes. The first-one detection chain is the most sensitive structure. A naive implementation creates a ripple-priority chain of length $2 \times \text{WIDTH}$. Synthesis tools partially mitigate this via tree balancing, but the chain still introduces a noticeably wider cone of logic between $\text{WIDTH}=8$ and $\text{WIDTH}=16$, visible as a $\sim 2.5\times$ cell count jump for only a $2\times$ growth in problem size.

3.3 Written Note

Why Round-Robin Enforces Fairness

A fixed-priority arbiter assigns a permanent rank to each requester. If higher-ranked agents are frequently active, lower-ranked ones can be deferred indefinitely — this is starvation. A round-robin arbiter removes the permanent rank: after each successful grant, priority rotates to the next index. Every requester that holds a request active is therefore guaranteed a grant within at most WIDTH cycles, regardless of what the others do. The worst-case latency is bounded, which is the formal definition of starvation freedom. The internal pointer encodes whose turn it is, resetting on reset and advancing only on a successful grant so idle cycles do not unfairly skip any requester.

Resource Scaling Observations

As shown in the synthesis table, the bottleneck is not state (FFs scale linearly) but the combinational priority-encoding logic. The doubled-vector trick used in this design ($\{\text{req}, \text{req}\}$ masked by a range starting at ptr) presents a $2 \times \text{WIDTH}$ -wide first-one chain to the synthesizer. The resulting cone of logic grows roughly as $O(\text{WIDTH} \times \log \text{WIDTH})$ in area. For $\text{WIDTH} = 4$ or 8 this is entirely acceptable for a single-cycle arbiter. At $\text{WIDTH} = 16$ the area is still small in absolute terms but the trend is concerning at scale.

Improvements for $\text{WIDTH} = 64$ or 128

At 64 or 128 inputs the single-level round-robin becomes impractical for single-cycle arbitration due to deep priority chains, excessive combinational area, and single-grant throughput limits. Four improvements are recommended:

- Hierarchical / tree-structured arbiter — Group inputs into clusters (e.g., 8-way), build a two-level tree with local arbiters per cluster and a global arbiter among cluster winners. Both levels are small and easy to time.
- Pipelined arbitration — Break logic into 2–3 pipeline stages (mask+encode in stage 1, grant register+pointer update in stage 2). Higher clock frequency at the cost of slightly longer response latency.
- Deficit Round Robin (DRR) — Assigns each requester a credit quota and tracks deficit, allowing bandwidth weighting while preserving the no-starvation guarantee. Hardware cost is one adder and comparator per requester per cycle.
- MUX-tree encoding — Replace the $\{\text{req}, \text{req}\}$ doubled-vector approach with a barrel-shift based on the pointer implemented as a $\log_2(\text{WIDTH})$ -deep MUX tree. More amenable to synthesis timing optimization than a linear priority chain.